Exercises on MapReduce

Exercise 1 (Similar to Exercise 2.3.1.(b) of [LRU14]) *Let S be a set of N integers represented by pairs (i, x_i), with $0 \leq i < N$, where i is the key of the pair and x_i is an arbitrary integer.*

1. *Design a 2-round MR algorithm to compute the arithmetic mean of the integers in S, using $O\left(\sqrt{N}\right)$ local space and $O(N)$ aggregate space.*
2. *Show how to modify the algorithm of the previous point to reduce the local space bound to $O\left(N^{1/4}\right)$, at the expense of an increased number of rounds.*
3. *Can you attain $M_L = O(M)$, for any $M < \sqrt{N}$?*

Solution.

1. The input-output specification of the problem is the following one:

   - **Input:** $S = \{(i, x_i) : 0 \leq i < N\}$.
   - **Output:** $(0, (1/N)\sum_{i=0}^{N-1} x_i)$.

   Observe that, in practice, in the output pair, key 0 would be omitted, unless it is needed for subsequent uses. While the trivial 1-round algorithm uses $\Theta(N)$ local space, the space requirements can be substantially lowered using the following simple 2-round strategy. We assume that the value N is a global variable, knwon to all map/reduce functions.

   Round 1

   - *Map phase*: map each pair (i, x_i) into the intermediate pair $(i \bmod \sqrt{N}, x_i)$.
   - *Reduce phase*: for each $j \in [0, \sqrt{N})$, let L_j be the list of values from the intermediate pairs (j, x), and map pair (j, L_j) into $(0, \text{sum}_j = \sum_{x \in L_j} x)$.

   Round 2

   - *Map phase*: empty.
   - *Reduce phase*: let L_0 be the list of values sum_j produced by Round 1, with $0 \leq j < \sqrt{N}$, and return pair $(0, (1/N)\sum_{\text{sum}_j \in L_0} \text{sum}_j)$.

   The map phase of Round 1 requires constant local space, while the reduce phases of Rounds 1 and 2 require $\Theta\left(\sqrt{N}\right)$ local space, since each reducer adds $\sqrt{N}$ integers. Hence, $M_L = \Theta\left(\sqrt{N}\right)$. As for the aggregate space, we have $M_A = \Theta(N)$, since in each round, $O(N)$ input, output and intermediate pairs of constant size are stored. $O(N)$.

2. The input-output specification is as before. The idea is to first to compute partial sums relative to groups of $N^{1/4}$ integers, and then to repeatedly aggregate the partial sums, in groups of size $N^{1/4}$. The pseudocode is given below. (We assume, for simplicity, that all quantities are integers.)

   Round 1.

   - *Map phase*: map each pair (i, x_i) into the intermediate pair $(i \bmod N^{3/4}, x_i)$.
   - *Reduce phase*: for each $j \in [0, N^{3/4})$, let L_j be the list of values from the intermediate pairs (j, x), and map pair (j, L_j) into $(j, \sum_{x \in L_j} x)$.

   Observation: At this point, there are $N^{3/4}$ partial sums to be further added, which are represented by pairs with distinct integer keys in $[0, N^{3/4})$.

   Round 2.

   - *Map phase*: map each pair (i, s) into the intermediate pair $(i \bmod N^{1/2}, s)$.
   - *Reduce phase*: for each $j \in [0, N^{1/2})$, let L_j be the list of values from the intermediate pairs (j, s), and map pair (j, L_j) into $(j, \sum_{s \in L_j} s)$.

   Observation: At this point, there are $N^{1/2}$ partial sums to be further added, which are represented by pairs with distinct integer keys in $[0, N^{1/2})$.

   Round 3.

   - *Map phase*: map each pair (i, s) into the intermediate pair $(i \bmod N^{1/4}, s)$.
   - *Reduce phase*: for each $j \in [0, N^{1/4})$, let L_j be the list of values from the intermediate pairs (j, s), and map pair (j, L_j) into $(0, \sum_{s \in L_j} s)$.

   Observation: At this point, only $N^{1/4}$ partial sums are left be further added, which are represented by pairs with key 0.

   Round 4.

   - *Map phase*: empty.
   - *Reduce phase*: let L_0 be the list of values s of the pairs produced by Round 3, and return pair $(0, (1/N) \sum_{s \in L_0} s)$.

   The map phases of Rounds 1, 2 and 3 requires constant local space, while the reduce phases of all rounds require $\Theta\left(N^{1/4}\right)$ local space, since each reducer adds $N^{1/4}$ integers. Hence, $M_L = \Theta\left(N^{1/4}\right)$. As for the aggregate space, we have $M_A = \Theta(N)$, since the number of input, output and intermediate pairs in each round is $O(N)$.

3. The above algorithm can be easily generalized to satisfy $M_L = O(M)$, for any given $M < \sqrt{N}$. To this purpose, we first compute the partial sums relative to groups of M integers, and then we repeatedly aggregate the partial sums, again in groups of

size M. Thus, Round 1 will transform the N input integers into N/M partial sums, and each Round i, with $i > 1$, will transform N/M^{i-1} partial sums into N/M^i partial sums. At the end of Round i, with $i = \log_M(N/M) = (\log_2 N/\log_2 M) - 1$, only M partial sums are left, and one extra round is sufficient to compute the final mean. So the number of rounds is $\log_2 N/\log_2 M$.

□

Exercise 2 (Similar to Exercise 2.3.1.(d) of [LRU14]) *Let S be a set of N integers represented by pairs (i, x_i), with $0 \leq i < N$, where i is the key of the pair and x_i is an arbitrary integer. We want to design an efficient MR algorithm to compute the number D of distinct integers in S.*

1. *Design a simple 2-round MR algorithm to compute D and analyze its local and aggregate space requirements. Under what circumstances is the local space proportional to N, thus not meeting the design goals of MR algorithms?*
2. *Design a better MR algorithm to compute D in $O(1)$ rounds, using $O\left(\sqrt{N}\right)$ local space and $O(N)$ aggregate space.*

Solution.

1. The input-output specification of the problem is the following one:

   - **Input:** $S = \{(i, x_i) : 0 \leq i < N\}$.
   - **Output:** $(0, D)$, where D is the number of distinct integers in S.

   We present a straightforward 2-round algorithm and a more space-efficient 4-round algorithm. The 2-round algorithm does a simple removal of duplicates as follows:

   Round 1.

   - *Map phase*: map each pair (i, x_i) into the intermediate pair (x_i, i), that is, x_i becomes the key.
   - *Reduce phase*: for each integer x, let L_x be the list of indices from the intermediate pairs $(x_i = x, i)$, and map pair (x, L_x) into $(0, x)$.

   Observation: At this point all duplicates have been removed.

   Round 2.

   - *Map phase*: empty.
   - *Reduce phase*: Let L_0 be the list of integers from all pairs $(0, x)$ produced by Round 1, and return $(0, D = |L|)$.

The aggregate space of the algorithm is $M_A = O(N)$, since in each round $O(N)$ input, output and intermediate constant-size pairs are processed. To analyze the local space, we define N_1 as the maximum number of occurrences of the same integer and recall that D denotes the number of distinct integers in the input set S. The map phase of Round 1 requires $O(1)$ local space, while the reduce phases of Rounds 1 and 2 require $\Theta(N_1)$ and $\Theta(D)$ local space, respectively. Hence, $M_L = \Theta(\max\{N_1, D\})$. Note that since $N_1 \cdot D \geq N$, we have $\max\{N_1, D\} \geq \sqrt{N}$, however, both N_1 and D can grow as large as N, asymptotically, hence M_L can be proportional to N in the worst case.

2. The following alternative strategy reduces the local space to the desired bound. (The input-output specification remains the same.) The idea is first to remove all duplicates working on subsets of size at most $\sqrt{N}$ each (Rounds 1 and 2), and then to count the number of surviving, and at this point distinct, integers, counting at most $\sqrt{N}$ of them at once (Rounds 3 and 4)

   Round 1.

   - *Map phase*: map each pair (i, x_i) into the intermediate pair $(i \bmod \sqrt{N}, x_i)$.
   - *Reduce phase*: for each key $j \in [0, \sqrt{N})$, let L_j be the list of values from the intermediate pairs (j, x), and map pair (j, L_j) into the set of pairs (j, x), one for every distinct integer $x \in L_j$.

   Observation: after Round 1, for every integer x there are at most $\sqrt{N}$ pairs (j, x), one for each value of $j \in [0, \sqrt{N})$.

   Round 2.

   - *Map phase*: map each pair (j, x) into the intermediate pair (x, j).
   - *Reduce phase*: for each x, let L_x be the list of indices j from the intermediate pairs (x, j), and map (x, L_x) into (x, j), where j is an index in L_x arbitrarily selected.

   Observation: after Round 2, for every integer x only one pair survives, and for each value of $j \in [0, \sqrt{N})$ there are at most $\sqrt{N}$ pairs with value j.

   Round 3.

   - *Map phase*: map each pair (x, j) into the intermediate pair (j, x).
   - *Reduce phase*: for each $j \in [0, \sqrt{N})$, let L_j be the list of values from the intermediate pairs (j, x), and map (j, L_j) into $(0, c_j = |L_j|)$. Note that c_j is a partial count of the distinct integers.

   Round 4.

   - *Map phase*: empty.

- *Reduce phase*: Let $L_0 = c_0, c_1, \ldots$ be the list of values of the pairs produced by Round 3, and return $(0, \sum_{c_j \in L_0} c_j)$ as final output.

It is easy to see that the above 4-round algorithm requires aggregate space $M_A = \Theta(N)$. As for the local space, the Map phase of each round requires constant local space, while the Reduce phases of all rounds require $O\left(\sqrt{N}\right)$ local space because of the initial subdivision into $\sqrt{N}$ partitions and of the fact that, after Round 1, at most $\sqrt{N}$ copies of each integer survive. Therefore, $M_L = O\left(\sqrt{N}\right)$.

The picture in Figure 1, describes the execution of the algorithm on a set S of $N = 16$ integers.

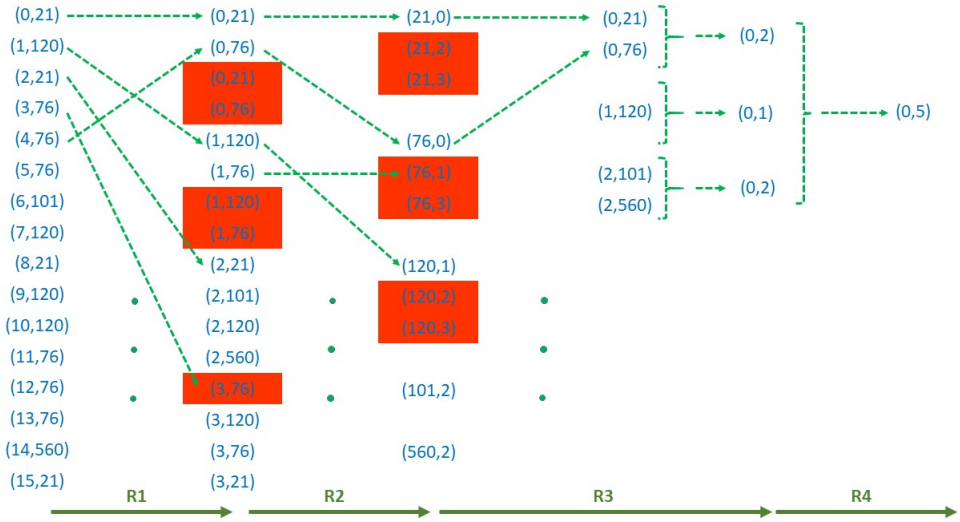

Figure 1: Execution of the algorithm on a set S of $N = 16$ integers

□

Exercise 3 *Design a 2-round MR algorithm for the word count problem, using random partitioning, and analyze its local and aggregate space requirements.*

Solution. The algorithm takes inspiration from the class count algorithm with random partitioning presented in class. Below is the pseudocode which uses ℓ random partitions.
Round 1.

- *Map phase*: map each document D_i into the set of pairs $(x, (w, c_i(w))$, one for every word $w \in D_i$, where $c_i(w)$ is the number of occurrences of w in D_i, and x is a random integer in $[0, \ell)$, chosen with uniform probability and independently for every word.

- *Reduce phase*: For each key $x \in [0, \ell)$, let L_x be the list of values (i.e., word-count pairs) from the intermediate pairs with key x. For each word w, occurring in L_x return the pair $(w, c(x, w))$, where $c(x, w)$ is the sum of the $c_i(w)$'s from pairs $(w, c_i(w))$ in L_x.

Round 2.

- *Map phase*: empty
- *Reduce phase*: for each word w, let L_w be the list of values $c(x, w)$ from the pairs produced by Round 1, and return the pair $(w, c(w) = \sum_{c(x,w) \in L_w} c(x, w))$. Note that $|L_w| \leq \ell$.

The analysis is similar to the one presented in class for the class count algorithm with random partitioning. Let us fix $\ell = \sqrt{N}$, where N is the total number of word occurrences in the various documents (i.e., the aggregate size of all documents). Let also $N_{\max}$ denote the maximum number of word occurrences in a document. Let m_x be the number of intermediate pairs with key x produced by the map phase of Round 1 (i.e., $m_x = |L_x|$), and let $m = \max_x m_x$. Note that the total number of intermediate pairs generated by the map phase is $\leq N$. It is easy to see that the above 2-round algorithm requires local space $M_L = O\left(N_{\max} + m + \sqrt{N}\right)$ and aggregate space $M_A = O(N)$. We can prove that $m = O\left(\sqrt{N}\right)$ with probability $\geq 1 - 1/N^5$ using exactly the same proof that was used for the theorem in the analysis of the class count algorithm with random partitioning (the details are left as an exercise). Hence, $M_L = O\left(N_{\max} + \sqrt{N}\right)$ with high probability. □

Exercise 4 *Design an $O(1)$-round MR algorithm for computing a matrix-vector product $W = A \cdot V$, where A is an $m \times n$ matrix, V is an n-vector, and $m \leq \sqrt{n}$. Your algorithm must use $o(n)$ local space and linear, that is, $O(mn)$ aggregate space.*

How would your solution change if m were larger?

Solution. The input-output specification of the algorithm is as follows. We assume that A is represented through the mn pairs $((i, j), A[i, j])$, with $0 \leq i < m$ and $0 \leq j < n$, where (i, j) is the key, and that V is represented through the n pairs $((-1, j), V[j])$, with $0 \leq j < n$, where $(-1, j)$ is the key. In this fashion we use the same representation for entries of A and V, and, in particular, we use keys from the same domain for both. As for W, we assume that in output it will be represented by the pairs $(i, W[i])$, with $0 \leq i < m$.

Note that $W[i]$ is the inner product of a row of A and V, which are both vectors of size n. A straightforward 1-round approach would be to make m copies of V in the map phase, and then, in the reduce phase, to compute $W[i]$, separately for $i \in [0, m)$, by gathering row i of A and copy i of V, and computing their inner product. However, this approach requires $\Theta(n)$ local space. In order to get $o(n)$ local space, we can break V and each row of A into $\sqrt{n}$ segments of size $\sqrt{n}$ each. In the first round, we compute separately the product of each segment of each row i of A and the corresponding segment of V, which yields $\sqrt{n}$

contributions to every $W[i]$. In the second round we just add up these $\sqrt{n}$ contributions, separately for every $W[i]$. The following algorithm implements this idea.

Round 1.

- *Map phase*: map each pair $((i,j), A[i,j])$, with $i \geq 0$, into the intermediate pair $((i, s = \lfloor j/\sqrt{n} \rfloor), (i, j, A[i,j]))$, and each pair $((-1,j), V[j])$ into the set of intermediate pairs $\{(i, s = \lfloor j/\sqrt{n} \rfloor), (-1, j, V[j])) : 0 \leq i < m\}$.

  Notation: we let $A^{(i,s)}$ denote segment s of row $A^{(i)}$, and let $V^{(i,s)}$ denote segment s of the i-th copy of V. Observe that both $A^{(i,s)}$ and $V^{(i,s)}$ contain (at most) $\sqrt{n}$ entries.

- *Reduce phase*: for each key (i,s), with $0 \leq i < m$ and $0 \leq s < \sqrt{n}$, let $L_{i,s}$ be the list of values from the intermediate pairs with key (i,s) (i.e., $L_{i,s}$ contains all entries of $A^{(i,s)}$ and $V^{(i,s)}$), and map $((i,s), L_{i,s})$ into the pair $(i, W_s[i] = A^{(i,s)} \circ V^{(i,s)})$. (The operator "$\circ$" denotes the inner product.)

Round 2.

- *Map phase*: empty

- *Reduce phase*: for each key i, let L_i be the list of all values $W_s[i]$ from the pairs $(i, W_s[i])$ produced by Round 1, with $0 \leq s < \sqrt{n}$, and return the output pair $(i, W[i] = \sum_{W_s[i] \in L_i} W_s[i])$.

For what concerns the local space, the map phase of Round 1 works on a single key value pair and transforms such a pair into 1 or $m \leq \sqrt{n}$ intermediate pairs, depending on whether it comes from A or V. The reduce phases of both rounds process sets of $O(\sqrt{n})$ entries each. Thus, we conclude that $M_L \in O(\sqrt{n})$. As for the aggregate space, A is not replicated, while the m replicas of V occupy overall $O(mn)$ space, that is, the same space as A. Thus, $M_A \in O(mn)$, that is linear in the input size.

If m were larger, the replication of each entry of V into m copies, which is performed in Round 1 of the above algorithm, would now require several rounds, where in each round every current copy is replicated $\sqrt{n}$ times. For example, if $\sqrt{n} < m \leq n$, two rounds would suffice. Once the entry of V have been replicated, the rest of the algorithm remains the same. The details are left as an exercise. □

Exercise 5 *Let P be a set of N* temperatures *measured by several sensors. Each element of P is a pair $(i, (s_i, temp_i))$, where $i \in [0, N-1]$ is a distinct integer key, s_i is a sensor, and $temp_i$ is a temperature. No assumptions are made on the number of pairs in P from the same sensor.*

1. *Let $K > 0$ be an integer constant. Design a 2-round MapReduce algorithm that, given P, returns a subset T of values $(s_i, temp_i)$ from pairs in P, selected as follows. For every sensor s, if in P there are $\leq K$ pairs $(i, (s_i, temp_i))$ with $s_i = s$, then T must contain the values of all these pairs, otherwise T must contain the values of exactly K of them, arbitrarily selected.*

2. *Analyze the local and aggregate space required by your algorithm. For full score, your algorithm must require $o(N)$ local space and linear aggregate space.*

3. *How does the local and aggregate space of your algorithm change if $K = \log_2 N$?*

Solution.

1. The 2-round MapReduce algorithm is the following.

   Round 1.

   - *Map phase*: map each pair $(i, (s_i, temp_i))$, into the intermediate pair $(i \bmod \sqrt{N}, (s_i, temp_i))$.
   - *Reduce phase*: for each key $0 \leq j < \sqrt{N}$, let L_j be the list of values (i.e., pairs $(s_i, temp_i)$), of intermediate pairs with key j. Return a subset $L'_j \subset L_j$ selected as follows. For every sensor s occurring in some pair of L_j, L'_j contains all $(s_i, temp_i)$ in L_j with $s_i = s$, if they are less the K, otherwise an arbitrary selection of K of them. Note that the output of the Reduce phase are pairs $(s, temp)$, where s is the key.

   Round 2.

   - *Map phase*: empty.
   - *Reduce phase*: for each sensor s, let L_s be the list of values (i.e., temperatures) of the output pairs of Round 1 with key s. From (s, L_s) return a set of pairs $(s, temp)$ with all distinct values $temp$ occurring in L_s, if these are less than K, otherwise an arbitrary selection of K of them.

2. In Round 1, there are at most $\sqrt{N}$ input pairs mapped to the same key j, hence to the same reducer, while in the reduce phase of Round 2 at most $K\sqrt{N}$ pairs gathered for each sensor s. Therefore, the required local space is $M_L = \Theta\left(K\sqrt{N}\right)$, which is $\Theta\left(\sqrt{N}\right)$, since K is constant. Also, throughout the algorithm data are only filtered and never replicated. Therefore, the required aggregate space is $M_A = \Theta(N)$.

3. If $K = \log_2 N$, then the previous analysis shows that the local space is $M_L = \Theta\left(K\sqrt{N}\right) = \Theta\left(\log N\sqrt{N}\right)$, while the aggregate space remains linear in N.

□